```
nastya@fedora:~/docker-lab$ sudo docker images myapp
                                                                    i Info →   U  In Use
IMAGE        ID             DISK USAGE   CONTENT SIZE   EXTRA
myapp:bad    569431ab9031       1.62GB          416MB    U
myapp:good   f94007a33054       81.9MB         19.6MB
nastya@fedora:~/docker-lab$
```

Команда «sudo docker images myapp» выводит информацию о Docker-образах с именем myapp. В выводе представлены два образа: myapp:bad и myapp:good. У них есть разница в размерах — myapp:bad занимает 1.62GB на диске, а myapp:good всего 81.9MB, при этом после распаковки при запуске контейнера bad образ занимает 416MB, а good — только 19.6MB. Разница в размерах объясняется тем, что myapp:bad собран с помощью неоптимизированного Dockerfile, который использует тяжелый базовый образ python:3.12, копирует все файлы без исключений и не применяет многоступенчатую сборку, что приводит к размеру 1.62GB. В то время как myapp:good — это «хороший образ», оптимизированный: он использует минимальный базовый образ, включает только необходимые бинарные файлы и зависимости.

```
CONTAINER ID   NAME       CPU %     MEM USAGE / LIMIT   MEM %     NET I/O           BLOCK I/O   PIDS
b70054544265   app-good   0.02%     20.58MiB / 128MiB   16.08%    52.2kB / 1.43kB   0B / 0B     1

^Cnastya@fedora:~/docker-lab$ sudo docker history myapp:good
[sudo] пароль для nastya:
IMAGE          CREATED       CREATED BY                                      SIZE      COMMENT
bbab993572bd   4 hours ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfil
e.v0
<missing>      4 hours ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfil
e.v0
<missing>      4 hours ago   USER appuser                                    0B        buildkit.dockerfil
e.v0
<missing>      4 hours ago   RUN /bin/sh -c adduser -D appuser # buildkit    24.6kB    buildkit.dockerfil
e.v0
<missing>      4 hours ago   ENV APP_VERSION=2.0                             0B        buildkit.dockerfil
e.v0
<missing>      4 hours ago   COPY app.py . # buildkit                        4.1kB     buildkit.dockerfil
e.v0
<missing>      4 hours ago   RUN /bin/sh -c pip install --no-cache-dir -r…   14.4MB    buildkit.dockerfil
e.v0
<missing>      4 hours ago   COPY requirements.txt . # buildkit              4.1kB     buildkit.dockerfil
e.v0
<missing>      4 hours ago   WORKDIR /app                                    0B        buildkit.dockerfil
e.v0
<missing>      3 weeks ago   CMD ["python3"]                                 0B        buildkit.dockerfil
e.v0
<missing>      3 weeks ago   RUN /bin/sh -c set -eux;  for src in idle3 p…   20.5kB    buildkit.dockerfil
e.v0
<missing>      3 weeks ago   RUN /bin/sh -c set -eux;   apk add --no-cach…   43.2MB    buildkit.dockerfil
e.v0
<missing>      3 weeks ago   ENV PYTHON_SHA256=c08bc65a81971c1dd578318282…   0B        buildkit.dockerfil
e.v0
<missing>      3 weeks ago   ENV PYTHON_VERSION=3.12.13                      0B        buildkit.dockerfil
e.v0
<missing>      3 weeks ago   ENV GPG_KEY=7169605F62C751356D054A26A821E680…   0B        buildkit.dockerfil
e.v0
<missing>      3 weeks ago   RUN /bin/sh -c set -eux;  apk add --no-cache…   3.62MB    buildkit.dockerfil
e.v0
<missing>      3 weeks ago   ENV LANG=C.UTF-8                                0B        buildkit.dockerfil
e.v0
<missing>      3 weeks ago   ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfil
e.v0
<missing>      8 weeks ago   CMD ["/bin/sh"]                                 0B        buildkit.dockerfil
e.v0
<missing>      8 weeks ago   ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /…   10.1MB    buildkit.dockerfil
e.v0
nastya@fedora:~/docker-lab$
```

Команда« sudo docker history myapp:good» выводит послойную историю сборки образа, где видна вся многоступенчатая структура: внизу находятся базовые слои от alpine Linux (10.1MB) и установка Python 3.12 (43.2MB), выше идут слои с копированием requirements.txt, установкой зависимостей pip с флагом --no-cache-dir (добавившими 14.4MB), затем копирование app.py (4.1kB),

создание непривилегированного пользователя appuser (24.6kB) и настройка рабочей директории.

```
nastya@fedora:~/docker-lab — sudo docker stats app-good
~/docker-lab
CONTAINER ID   NAME       CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
b70054544265   app-good   0.02%     20.58MiB / 128MiB   16.08%    19kB / 1.43kB   0B / 0B     1
```

Команда «sudo docker stats app-good» отображает статистику использования ресурсов работающего контейнера в реальном времени. Контейнер работает с заданными при запуске ограничениями: лимит памяти установлен в 128 MiB, и контейнер использует всего 20.58 MiB, что составляет 16.08% от выделенного лимита. Загрузка центрального процессора минимальна и составляет всего 0.02%. Сетевой ввод-вывод показывает 19 kB полученных и 1.43 kB отправленных данных, операции с блочными устройствами отсутствуют, и внутри контейнера запущен всего один процесс.

```
Username: nastasya111
nastya@fedora:~/docker-lab$ sudo docker push nastasya111/flask-demo:v1.0
The push refers to repository [docker.io/nastasya111/flask-demo]
41418446d7e5: Pushed
6f3f43492c5b: Pushed
9cdb42597c5a: Pushed
6de7811bee64: Pushed
589002ba0eae: Pushed
298a7d7a81d9: Pushed
89a8227ba99b: Pushed
c43f646c64f4: Pushed
9beaea79ae55: Pushed
1ee3baf44600: Pushed
v1.0: digest: sha256:bbab993572bdb16995ded48202e75c7a483569dcd30ba7b48b08bbf53a299b06 size: 856
nastya@fedora:~/docker-lab$ docker rmi nastasya111/flask-demo:v1.0
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
nastya@fedora:~/docker-lab$ sudo docker rmi nastasya111/flask-demo:v1.0
Untagged: nastasya111/flask-demo:v1.0
nastya@fedora:~/docker-lab$
```

Команда «sudo docker push nastasya111/flask-demo:v1.0» загружает образ в мой репозиторий (nastasya111). В процессе публикации видно, как поочередно отправляются слои образа — всего 10 слоев с уникальными идентификаторами, что подтверждает использование оптимизированной многоступенчатой сборки, где каждый слой кэшируется и переиспользуется. После успешной отправки выводится digest образа, который является уникальной криптографической суммой, идентифицирующей точное содержимое образа, и размер 856 байт.

URL моего образа на Docker Hub будет: https://hub.docker.com/r/nastasya111/flask-demo. В формате для использования в командах Docker это docker.io/nastasya111/flask-demo:v1.0, где nastasya111 — имя пользователя, flask-demo — название репозитория, а v1.0 — тег версии.